Interview Questions

Companies	Google · Meta · Amazon · Microsoft · Apple · Netflix · Uber · Airbnb
Levels	Staff / Senior / L5–L7 / E5–E7
Covers	Functional Reqs · Scale Estimates · Architecture · Deep-dives · Trade-offs

Table of Contents

Each question includes: Requirements → Scale Estimates → High-Level Design → Component Deep-Dives → Trade-offs → Interviewer Follow-ups

1 Design a URL Shortener

Google · Amazon · Meta | Medium

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Shorten a long URL to a 7-char alias. Redirect short URL to original. Optional custom alias. Link expiration. Analytics (click count, geo).
Non-Functional	100M URLs created/day. 10:1 read:write ratio. 1B redirects/day. Redirect latency <10ms (P99). High availability (99.99%). No data loss. Idempotent creates.
Out of Scope	User accounts, billing, A/B testing, full analytics dashboard.

SCALE ESTIMATES

Back-of-Envelope

Metric	Calculation	Result
Writes/sec	100M / 86,400	~1,160 URLs/sec
Reads/sec	1B / 86,400	~11,600 redirects/sec
Storage (5yr)	100M × 365 × 5 × 500B	~90 TB
Short URL space	62^7	~3.5 trillion unique keys

HIGH-LEVEL ARCHITECTURE

Core Components

```
Client --> Load Balancer --> Shortener Service --> DB (MySQL / Cassandra)
|
Key Generation Service (KGS)
|
Cache (Redis) <--- Redirect Service --> Analytics (Kafka + ClickHouse)
```

DEEP DIVE

Key Generation Strategy

Two approaches — pick based on interview context:

Hash + collision	MD5(longURL) → take first 7 chars. Problem: collision. Fix: append counter, re-hash. Simple but needs uniqueness check.
Pre-generated Keys (KGS)	Background worker pre-generates 7-char keys, stores in "unused" table. Redirect service moves to "used" on claim. Solves contention. Used by TinyURL in practice.

Snowflake ID → Base62	Generate 64-bit monotonic ID via Snowflake, encode to Base62. Provides ordering + no collisions. Preferred for distributed systems.

DATA MODEL

Schema

```
urls table
short_code CHAR(7) PRIMARY KEY
long_url TEXT NOT NULL
user_id BIGINT
created_at TIMESTAMP
expires_at TIMESTAMP
click_count BIGINT DEFAULT 0
```

TRADE-OFF: Custom short codes (vanity URLs) create hot-key risk in sharded DB. Shard by short_code hash, not alphabetically.

DEEP DIVE

Redirect Flow & Caching

Cache strategy	Cache-aside. Redis TTL matches URL expiry. LRU eviction. 80/20 rule: top 20% of URLs get 80% of traffic.
Cache size	11,600 reads/sec × 500B × 20% hot URLs × 24h = ~10 GB RAM
Redirect type	301 (Permanent) → browser caches; fewer server hits. 302 (Temporary) → every request hits server; better analytics.

INTERVIEWER TIP: Always ask the interviewer whether analytics matter — it drives the 301 vs 302 decision.

2 Design a Social Media News Feed

Meta · Twitter · LinkedIn | Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	User posts content (text, images, video). User sees feed of posts from people they follow, ranked by relevance/recency. Infinite scroll / pagination.
Non-Functional	500M DAU. 1M posts/day. Feed load <500ms (P99). Eventual consistency OK. 99.9% uptime. Handle celebrities with 100M+ followers.

SCALE ESTIMATES

Back-of-Envelope

Metric	Value
Feed reads/sec	500M DAU × 10 reads/day / 86,400 = ~58,000/sec
Posts/sec	1M / 86,400 = ~12 writes/sec
Fan-out celebrity (10M followers)	12 posts/sec × 10M = 120M writes/sec peak
Storage (media)	1M posts/day × 1 MB avg = 1 TB/day

HIGH-LEVEL ARCHITECTURE

Feed Generation Approaches

Approach	Description	Pros	Cons
Pull (Fan-out on Read)	Fetch posts from followed users at read time	Low write cost	Slow reads, heavy DB
Push (Fan-out on Write)	Write to each follower's feed cache on post	Fast reads, simple	Write amplification for celebrities
Hybrid (Meta / Twitter)	Push for normal users; Pull for celebrities	Best of both	Complex ranking merge

DEEP DIVE

Hybrid Fan-out Architecture

```
Post Service --> Message Queue (Kafka)
|
Fan-out Worker
/ \
Regular users (<5K followers) Celebrities (>5K followers)
```

```
Write to Redis feed cache Store in Post DB only
|
Feed Assembly Service (at read time)
merges cached feed + celebrity posts
```

Feed Cache (Redis)	Sorted Set keyed by user_id. Score = timestamp. Store post_id only, not full content. Max 500 entries per user feed.
Ranking	EdgeRank-style: affinity × weight × time_decay. ML models at scale.
Pagination	"Cursor-based" using timestamp, not OFFSET. OFFSET is O(n) in SQL.
Media Storage	CDN + Object Store (S3). Never store images in DB. Use pre-signed URLs.

TRADE-OFF: Storing post_id in feed cache means a second DB lookup per post. Trade-off: cache size (store full content) vs consistency (invalidation on edit).

3 Design a Distributed Cache (Redis-like)

Amazon · Google · Uber | Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	get(key), set(key, value, ttl), delete(key). Optional: LRU/LFU eviction, pub/sub, atomic increment, persistence.
Non-Functional	<1ms P99 latency. 99.99% availability. Horizontal scale to 100+ nodes. Support 1M ops/sec. Tunable consistency (strong vs eventual).

CORE DESIGN

Data Partitioning

Strategy	Description	Trade-off
Mod Hashing	key % N nodes	Simple; reshuffling all keys on scale event
Consistent Hashing	Key ring; only K/N keys move on add/remove	Standard choice
Consistent + Virtual Nodes	Each physical node has 150+ virtual nodes	Balanced load; used by Redis Cluster

DEEP DIVE

Replication & Consistency

```
Primary ■■writes■■> Replica 1
■■writes■■> Replica 2

Write modes:
WRITE_ONE – ack after primary write. Fast. Risk of data loss on crash.
WRITE_QUORUM– ack after majority (N/2+1). Balanced. Redis default recommendation.
WRITE_ALL – ack after all replicas. Slowest. Strongest durability.
```

Eviction Policies	LRU (Least Recently Used) – standard. LFU (Least Frequently Used) – better for Zipfian distributions. noeviction – return error when full.
Persistence	RDB snapshots (point-in-time, fast recovery). AOF log (every write, slower but no data loss). Hybrid: RDB + AOF tail.
Cluster Coordination	Raft or ZooKeeper for leader election. Gossip protocol for node membership and failure detection (used by Redis Sentinel).
Cache Patterns	Cache-aside (app manages), Write-through (sync to DB), Write-behind (async), Read-through (cache manages misses).

TRADE-OFF: Write-through adds latency on every write but prevents cache stampede. Write-behind is faster but risks data loss in crash window.

4 Design a Rate Limiter

Stripe · Cloudflare · Uber | Medium

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Throttle API calls per user / IP / API key. Configurable rules. Return 429 with Retry-After header. Differentiate per endpoint.
Non-Functional	<5ms added latency. Distributed (no single node). 99.99% uptime. Rules changeable without deploy. 100K rules.

ALGORITHMS

Rate Limiting Algorithms Compared

Algorithm	How It Works	Pros	Cons
Token Bucket	Tokens refill at fixed rate; consumed per request	Handles bursts, smooth	Two counters per user
Leaky Bucket	Queue drains at constant rate	Smooth output	Doesn't handle bursts
Fixed Window	Count resets every N seconds	Simple	Boundary spike (2x traffic at edge)
Sliding Window Log	Timestamps of requests in sorted set	Precise	High memory
Sliding Window Counter	Interpolation of prev + curr window	Accurate, low memory	Slightly approximate

DEEP DIVE

Distributed Implementation with Redis

```
-- Sliding Window Counter in Redis Lua (atomic) --
local key = KEYS[1]
local now = tonumber(ARGV[1]) -- unix ms
local window = tonumber(ARGV[2]) -- ms
local limit = tonumber(ARGV[3])

redis.call("ZREMRANGEBYSCORE", key, 0, now - window)
local count = redis.call("ZCARD", key)
if count < limit then
redis.call("ZADD", key, now, now)
redis.call("EXPIRE", key, window / 1000)
return 1 -- allowed
end
return 0 -- rejected
```

Architecture	Client → API Gateway (rate-limit check) → Redis Cluster → Backend. Rules stored in a config service (etcd/Consul) — hot-reload.
Headers	X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset, Retry-After
Soft vs Hard	Soft limits: warn but allow. Hard limits: reject with 429. Shadow mode: log violations without rejecting (safe rollout).
Race condition	Use Redis MULTI/EXEC or Lua scripts for atomic read-increment-compare.

INTERVIEWER TIP: Mention that rate limiter should be in API gateway, not app tier, to protect backend from even reaching application code.

5 Design a Push Notification System

Meta · Apple · Google | Medium-Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Send push (iOS/Android), SMS, email notifications. User-level opt-in/out settings. Retry on failure. Delivery status tracking. Templating.
Non-Functional	10M notifications/day. Delivery within 10s of trigger. At-least-once delivery. Support for scheduled and event-driven sends.

HIGH-LEVEL ARCHITECTURE

Notification Pipeline

```
Trigger Sources (Product Services, Schedulers, Marketing Tools)
|
Notification Service API
|
Message Queue (Kafka) ← fan-out by channel
/ | \
Push Worker SMS Worker Email Worker
| | |
APNs/FCM Twilio/SNS SendGrid/SES
| | |
Delivery Status DB (Cassandra)
```

Device Token Management	Store iOS (APNs token) and Android (FCM token) per user. Tokens expire/rotate — handle InvalidToken errors by removing from DB.
Retry Strategy	Exponential backoff with jitter. Dead-letter queue (DLQ) for permanent failures. Max 3 retries over 1 hour.
Deduplication	Idempotency key per notification. Redis SET with TTL before send. Prevents duplicate sends on retry.
User Preferences	Stored in Redis (read-heavy). Respect DND windows, channel opt-outs, frequency caps (max 5/day per user).
Templating	Mustache / Handlebars templates. Personalisation variables injected at send time. Support i18n locale.

TRADE-OFF: At-least-once delivery is standard (idempotency on receiver). Exactly-once would require distributed transactions across APNs/FCM — impractical.

6 Design a Distributed Web Crawler

Google · Amazon · Microsoft | Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Crawl the entire web starting from seed URLs. Store raw HTML. Extract and follow links. Respect robots.txt. Recrawl on schedule.
Non-Functional	1B pages in 4 weeks. 40K pages/sec peak. Handle 5B+ URLs without revisiting. Politeness: max 1 req/domain/sec. Handle trap/cyclic links.

SCALE ESTIMATES

Back-of-Envelope

Metric	Value
Pages/sec needed	1B / (28 days × 86400) ≈ 413/sec → 2× safety = ~1000/sec
HTML storage (avg 100KB)	1B × 100KB = 100 TB
URL frontier size	5B URLs × 64B each = ~320 GB
DNS cache size	20M domains × 100B = 2 GB

HIGH-LEVEL ARCHITECTURE

Core Components

```
Seed URLs
|
URL Frontier (Priority Queue + Politeness Queue per domain)
|
Fetcher Workers (1000s, distributed)
|
HTML Parser --> URL Extractor --> URL Seen? (Bloom Filter)
| |
Content Store (S3) Add to Frontier (if new)
|
Link Graph DB (Neo4j / custom)
```

URL Frontier	Priority queue (BFS with PageRank hints). Per-domain politeness queues ensure crawl delay. Use Redis sorted sets.
Deduplication	Bloom filter for URL seen check (false-positive OK). SimHash / MinHash for near-duplicate content detection.
DNS Cache	Cache DNS resolutions for ~1M domains. Refresh every 24h. Prevents DNS bottleneck (DNS is 10ms per lookup).

Robots.txt	Cache robots.txt per domain. Parse Disallow rules. Respect Crawl-delay directive. Refresh weekly.
Trap Detection	Detect spider traps: URL length limit (2KB), depth limit (50 hops), domain request cap (100K pages/domain).
Storage	Raw HTML in object store (S3). Index metadata in Elasticsearch. URL-to-content hash for dedup.

TRADE-OFF: Breadth-first crawl is fair but misses deep important pages. Weighted BFS using PageRank scores in priority queue is standard (Googlebot approach).

7 Design a Ride-Sharing Service (Uber / Lyft)

Uber · Lyft · DoorDash | Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Rider requests ride → system matches nearest available driver. Real-time driver location tracking. Dynamic pricing (surge). Trip lifecycle: request → accept → pickup → in-trip → complete.
Non-Functional	1M active trips at peak. Driver location update every 5 sec. Match latency <1 min. Location query P99 <100ms. High availability.

SCALE ESTIMATES

Back-of-Envelope

Metric	Value
Active drivers	500K at peak
Location updates/sec	500K drivers × 1/5sec = 100K writes/sec
Location storage (24h)	100K/sec × 28B per record × 86400 = ~240 GB/day
Geospatial query radius	2 km search, ~50 drivers in dense city

DEEP DIVE

Geospatial Matching

GeoHash	Encode lat/lng to Base32 string. Nearby cells share prefix. Stored in Redis (GEOADD command). Variable precision (6 chars = ~1.2km cell).
H3 Hexagons	Uber's open-source hierarchical hexagonal grid. Consistent cell area unlike GeoHash rectangles. Used for surge pricing zones.
QuadTree	Recursively partition 2D space. Dynamic density handling. Good for offline map indexing. Complex for real-time updates.
S2 Cells	Google's spherical geometry library. Used in Google Maps. Hierarchical, accurate for long-distance queries.

HIGH-LEVEL ARCHITECTURE

Trip Lifecycle Services

```
Rider App → API Gateway → Ride Request Service → Match Engine
|
Location Service ← Driver App (every 5s)
```

```
(Redis Geo + Time-series DB)
|
Find top-N nearest available drivers
|
Supply/Demand → Surge Pricing Engine
|
Send offer to best driver (ETA + earnings)
|
Driver accepts → Trip Service (state machine)
→ Notify Rider → GPS tracking via WebSocket
```

TRADE-OFF: Broadcasting ride requests to multiple drivers simultaneously (Uber) vs sequential offering (Lyft) — broadcast is faster to fill but burns driver attention.

8 Design a Distributed Message Queue (Kafka-like)

LinkedIn · Confluent · Meta | Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Producers publish messages to topics. Consumers read messages in order per partition. Consumer groups for fan-out. Message retention. Replay.
Non-Functional	1M msgs/sec throughput. Message durability (no loss). Ordered delivery per partition. Horizontal scale. <10ms P99 latency.

CORE CONCEPTS

Architecture Primitives

Concept	Description
Topic	Logical stream of messages (e.g., user-events, payments)
Partition	Ordered, immutable log within a topic. Unit of parallelism.
Offset	Sequential ID per message within a partition. Consumer tracks offset.
Broker	Server storing partitions. Leader handles reads+writes; followers replicate.
Consumer Group	Each partition assigned to exactly one consumer in group. Enables parallelism.
ISR (In-Sync Replicas)	Subset of replicas up-to-date with leader. Leader only acks after ISR confirm.

DEEP DIVE

Write Path & Durability

```
Producer
|-- partitioner(key) --> Partition 0 Leader (Broker 1)
|-- replicate --> Follower (Broker 2)
|-- replicate --> Follower (Broker 3)
|
Wait for ISR acks (acks=all)
Append to segment file (sequential write = fast)
Update offset index
<--- ack sent to Producer
```

Storage	Append-only log files on disk. Sequential I/O = 500MB/s vs 50MB/s random. Data retention by time or size (default 7 days).

Zero-copy	sendfile() syscall bypasses kernel buffer copy. Critical for high throughput (Kafka's secret weapon).
Consumer Pull	Consumers pull at their own pace. No backpressure complexity. Long-poll with timeout for low-latency.
Leader Election	ZooKeeper (legacy) or KRaft (Kafka 3.x, Raft-based). Controller manages partition leader assignments.
At-least-once	Default. Producer retries; consumer re-processes on restart. Exactly-once via idempotent producer + transactional API.

TRADE-OFF: More partitions = more parallelism but more open file handles, longer leader election, more metadata overhead. Rule: 10-50 partitions per topic.

9 Design a Video Streaming Platform (YouTube / Netflix)

Google · Netflix · Amazon | Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Upload video. Transcode to multiple resolutions. Stream with adaptive bitrate. Search/discover. Comments, likes, subscriptions. View count.
Non-Functional	500 hours of video uploaded per minute (YouTube scale). 2B monthly users. Smooth playback on slow networks. <2s startup time. 99.9% availability.

SCALE ESTIMATES

Back-of-Envelope

Metric	Value
Upload bandwidth	500 hr/min × 1 Gbps avg = 30 Tbps ingress
Storage (raw)	500 hr/min × 60min × 2GB/hr = 60 TB/day raw
Storage (transcoded)	60 TB × 5 resolutions × compression ≈ 100 TB/day
CDN egress	2B users × 1hr/day × 5 Mbps = 10 Pbps/day

HIGH-LEVEL ARCHITECTURE

Upload & Streaming Pipeline

```
Upload Flow:
Creator → Resumable Upload Service → Raw Object Store (S3)
|
Transcoding Service (FFmpeg workers)
[360p, 480p, 720p, 1080p, 4K]
|
CDN Origin → Edge CDN Nodes (PoPs)

Playback Flow:
Viewer → CDN Edge (HLS/DASH manifest) → adaptive client player
Player requests next segment → CDN serves from nearest PoP
Bitrate ladder switch on bandwidth change (ABRS algorithm)
```

Adaptive Bitrate	HLS (Apple) / DASH (Google): manifest file lists segments at each quality. Player monitors bandwidth → switches quality every 2-10s segment.
Transcoding	GPU instances for H.264/H.265/AV1 encoding. Job queue (SQS). Scene detection for smart keyframe placement.

Storage Tiering	Hot content (top 5%) on CDN. Warm (past 30 days) in regional object store. Cold (old/rare) in Glacier / archival storage.
Metadata DB	Video metadata in MySQL (title, description, channel). View counts in Redis (high write) → flush to DB every 5 min.
Search	Elasticsearch for title/description search. Video embeddings for semantic search. Recommendation ML pipeline.

TRADE-OFF: Pre-transcoding all resolutions wastes storage for never-watched videos. Netflix uses dynamic optimizer: only transcode popular resolutions on demand initially.

10 Design a Search Autocomplete System

Google · Bing · Amazon | Medium-Hard

REQUIREMENTS

Functional & Non-Functional Requirements

Functional	Return top-5 search suggestions as user types. Suggestions based on global search popularity. Support Unicode. Filter adult/hateful content.
Non-Functional	<100ms P99 latency at keypress. 10M QPS (Google scale). Data freshness: suggestions updated within 1 week of trend. Highly available — degraded mode if data stale.

DATA STRUCTURE

Trie + Frequency Scoring

Approach	Description	Trade-off
Trie	Prefix tree; each node stores top-5 completions	Fast prefix lookup; large memory
Trie + DFS	Walk trie, DFS to find all completions, rank by freq	Flexible but O(P) not O(1)
Trie with cached top-k	Each node pre-stores its top-k results	O(1) lookup; expensive updates
Ternary Search Tree	Memory-efficient alternative to trie	Slower; less common in interviews

DEEP DIVE

System Architecture

```
Data Collection Layer (offline):
Search logs → Kafka → Spark Aggregation (weekly) → Frequency DB
|
Build / Update Trie
Serialize to disk
Shard by prefix (a-f, g-m, n-z, 0-9)

Query Layer (online):
User types "app" → API Gateway → Autocomplete Service
→ Route to correct trie shard (prefix "a")
→ Trie node lookup → return cached top-5
→ CDN caches popular prefixes (covers 80%+ of queries)
```

Trie Updates	Immutable tries: build new trie offline, swap pointer atomically. Avoids locking. Build frequency: weekly for full rebuild, hourly incremental for trending.

Sharding	Partition trie by first 2 characters. Avoids hot-shard issue (e.g., "th" prefix is heavily hit). Consistent hashing for rebalance.
CDN Caching	Top 1000 prefixes cover ~80% of queries. Cache at CDN edge. TTL = 1 hour. Huge infrastructure savings.
Personalization	Blend global top-k with user's recent searches. Weighted sum: 70% global + 30% personal. Stored per-user in Redis.
Filtering	Blocklist stored as hash set in trie builder. Filter at build time, not query time. Near-zero latency impact.

INTERVIEWER TIP: Always mention CDN caching for autocomplete — it's the single biggest win and interviewers love seeing candidates think about read-heavy query patterns.

System Design Cheat Sheet

Back-of-Envelope Rules of Thumb

Rule	Value
Seconds in a day	86,400 (~100K)
1M req/day	~12 req/sec
1B req/day	~12,000 req/sec
SSD random read	~100 microseconds
SSD sequential read	~1 GB/s
Network round trip	Same DC: 0.5ms | Cross-region: 100ms
Mutex lock/unlock	~25 nanoseconds
Branch mispredict	~5 nanoseconds

Component Selection Quick Guide

Need	Use This
SQL + ACID transactions	PostgreSQL / MySQL
High write throughput, flexible schema	Cassandra / DynamoDB
Full-text search	Elasticsearch / Solr
Graph relationships	Neo4j / Amazon Neptune
Caching / leaderboards / geo	Redis
Time-series data	InfluxDB / Prometheus / TimescaleDB
Message streaming	Apache Kafka / AWS Kinesis
Task queues	RabbitMQ / SQS / Celery
Object / blob storage	S3 / GCS / Azure Blob
Content delivery	CloudFront / Akamai / Fastly
Service discovery	Consul / etcd / ZooKeeper
Distributed tracing	Jaeger / Zipkin / AWS X-Ray

CAP Theorem & Common Trade-offs

System	CAP Choice	Notes
Cassandra	AP	Eventual consistency; tunable quorum

HBase	CP	Strong consistency; ZooKeeper coordination
Dynamo/DynamoDB	AP	Eventual by default; optional strong reads
Spanner	CP	Globally consistent; TrueTime API
Redis	CP	Primary is source of truth; async replication
Kafka	CP	ISR-based durability; leader-epoch fencing

Numbers Every Engineer Should Know

Latency	Time
L1 cache reference	1 ns
L2 cache reference	4 ns
Main memory access	100 ns
SSD random read	100 us
HDD seek	10 ms
Packet NYC → CA	40 ms
Packet NYC → Amsterdam	80 ms